

Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Fundamentos de Sistemas Computacionales

RADAR 2D

Documento Técnico — Proyecto 2

Estudiantes

Esteban Alejandro Sánchez Ledezma

Dominick Robles

Profesor: Leonardo Andrés Araya Martínez

I Semestre 2026

Índice

Índice	2
Introducción	3
Conclusiones	4
Recomendaciones	5
Análisis de Resultados	6
Precisión del sensor y rango efectivo	6
Barrido del servo y sincronización	6
Comunicación serial y threading	6
Tracker multi-objeto y velocidad	6
Predicción de trayectoria parabólica	6
Bibliotecas Utilizadas	7
Python (GUI y lógica de procesamiento)	7
Arduino (firmware del hardware)	7
Diagramas	8
Fotos del sistema funcionando	8
Referencias y Fuentes Consultadas	10
Desarrollo de Software e Interfaz Gráfica (Python / Tkinter)	10
Hardware y Arduino	10
Diagramas y Diseño Lógico	10
Gestión del Proyecto y Control de Versiones	10
Referencias y Consultas con la IA	11
Chat 1 — Hilo serial y bloqueo de Tkinter	11
Chat 2 — Puerto serial ocupado por el IDE de Arduino	11
Chat 3 — Detección automática del puerto Arduino	11
Chat 4 — Conversión de coordenadas polares a cartesianas	11

Introducción

El presente proyecto consiste en el desarrollo de un sistema de radar bidimensional (2D) funcional, diseñado como parte del curso Fundamentos de Sistemas Computacionales del Tecnológico de Costa Rica. La solución integra hardware programable con una interfaz gráfica de usuario, permitiendo la detección, visualización y seguimiento de objetos en tiempo real.

El sistema se compone de dos componentes principales: un módulo de hardware basado en un Arduino UNO que controla un servomotor de 180° y un sensor ultrasónico HC-SR04, y un módulo de software desarrollado en Python que recibe los datos a través del puerto serial, los procesa matemáticamente y los despliega en una interfaz gráfica construida con Tkinter.

Entre las funcionalidades implementadas se encuentran la conversión de coordenadas polares a cartesianas, el cálculo de velocidad de los objetos detectados mediante el delta de posición entre lecturas, el rastreo simultáneo de múltiples objetos utilizando distancia euclidiana, y la predicción de trayectoria parabólica. Todo esto se visualiza en tiempo real mediante una interfaz de estilo HUD (Heads-Up Display) con estética de radar militar.

En este documento se presentan las conclusiones del proyecto, las recomendaciones para futuras mejoras, el análisis de los resultados obtenidos, las bibliotecas y herramientas utilizadas, los diagramas del sistema y las referencias consultadas.

Conclusiones

El desarrollo del proyecto permitió aplicar de manera integrada y práctica conocimientos de sistemas embebidos, electrónica digital, programación orientada a objetos en Python y comunicación serial.

Se logró implementar un sistema funcional capaz de detectar objetos en un barrido de 180°, transmitir los datos al computador en tiempo real y representarlos de forma gráfica mediante una interfaz de radar estilo HUD. La arquitectura de módulos independientes (matemáticas, comunicación, GUI) facilitó el desarrollo incremental y la depuración aislada de cada componente.

La implementación del hilo daemon para la lectura serial demostró ser fundamental para mantener la fluidez de la interfaz gráfica. Sin dicha separación, el mainloop de Tkinter se bloquearía ante cualquier espera de datos por parte del puerto serial, haciendo el sistema inutilizable.

El tracker multi-objeto basado en distancia euclidiana resultó efectivo dentro del rango de 100 cm del sensor, logrando asociar correctamente las lecturas a objetos ya conocidos cuando su desplazamiento entre barridos consecutivos se mantenía por debajo del umbral definido de 15 cm.

Finalmente, el proyecto evidenció la importancia de coordinar adecuadamente las responsabilidades de software y hardware: el Arduino se encargó exclusivamente de capturar y enviar datos crudos, mientras que toda la lógica de procesamiento, cálculo y visualización residió del lado del computador, siguiendo el principio de separación de responsabilidades.

Recomendaciones

Se recomienda realizar pruebas individuales de cada módulo del sistema antes de la integración total, especialmente del módulo de comunicación serial, ya que los conflictos de puerto entre el IDE de Arduino y el programa en Python son una fuente común de errores difíciles de diagnosticar.

Es aconsejable ajustar el parámetro `DELAY_SERVO_MS` del sketch de Arduino según las características físicas del servomotor utilizado. Un valor muy bajo provoca que el sensor tome lecturas antes de que el servo llegue a la posición deseada, introduciendo imprecisiones en el ángulo reportado y por ende en la posición cartesiana calculada.

Para mejorar la estabilidad de las lecturas del sensor HC-SR04 se recomienda implementar un filtro de promedio móvil o de mediana sobre las últimas N lecturas del mismo ángulo, lo que reduciría significativamente el impacto del ruido ultrasónico y los falsos positivos generados por reflexiones en superficies irregulares.

Se sugiere también añadir una calibración inicial al arrancar el sistema, donde el Arduino realice un barrido completo de 0° a 180° y de vuelta antes de comenzar a enviar datos, garantizando que el estado inicial del servo y del tracker estén sincronizados.

Para futuras versiones, se podría explorar la implementación de un radar 3D utilizando un segundo servomotor en el eje vertical, tal como lo sugiere el enunciado del proyecto. Esto requeriría ajustes en el protocolo serial para transmitir tres valores (ángulo horizontal, ángulo vertical, distancia) y en la lógica matemática para trabajar con coordenadas esféricas.

Análisis de Resultados

Durante el desarrollo y las pruebas del sistema se evaluaron tres aspectos principales: la precisión en la detección de objetos, la respuesta del hardware durante el barrido continuo y la correcta comunicación entre los módulos de software.

Precisión del sensor y rango efectivo

El sensor ultrasónico HC-SR04 mostró lecturas confiables dentro del rango de 2 cm a 100 cm, que es el rango configurado en el sketch de Arduino mediante la constante `DISTANCIA_MAXIMA_CM`. Fuera de este rango el hardware reporta -1 como valor de distancia, dato que el software descarta correctamente antes de procesarlo. Se observó que en superficies blandas o con ángulos pronunciados respecto al haz ultrasónico, el sensor puede producir lecturas erróneas o ausencia de eco; esta situación se maneja en el sketch verificando si `pulseIn` retorna 0 dentro del timeout configurado.

Barrido del servo y sincronización

El servomotor realiza un barrido continuo de ida y vuelta entre 0° y 180° con pasos de 2° y una espera de 40 ms entre posiciones. Este intervalo fue ajustado durante las pruebas para encontrar un balance entre velocidad de barrido y tiempo de estabilización mecánica del servo, ya que valores menores a 30 ms provocaban que el sensor tomara lecturas mientras el servo aún estaba en movimiento. El barrido completo de 180° tarda aproximadamente 3.6 segundos en completarse.

Comunicación serial y threading

La comunicación entre el Arduino y la GUI de Python se realiza por puerto serial a 9600 baudios, en el formato "ángulo,distancia" por línea. Para evitar que la espera bloqueante de `readline()` congele la interfaz gráfica, se implementó la clase `LectorSerial` que corre en un hilo daemon independiente y deposita cada lectura en una `queue.Queue` thread-safe. La GUI consume esta cola periódicamente mediante llamadas `after()` de `Tkinter`, garantizando que la animación del radar se mantenga fluida independientemente de la velocidad de llegada de los datos del Arduino.

Tracker multi-objeto y velocidad

El tracker implementado con distancia euclidiana logró seguir correctamente hasta tres objetos simultáneos en las pruebas realizadas. La velocidad se calcula usando la fórmula $v = d/t$, donde d es la distancia euclidiana recorrida en el plano cartesiano entre dos lecturas consecutivas del mismo objeto y t es el delta de tiempo entre dichas lecturas. Los objetos sin actualización por más de 3 segundos (`TIMEOUT_SEGUNDOS`) son eliminados automáticamente del estado del tracker.

Predicción de trayectoria parabólica

La predicción de trayectoria se implementó usando las ecuaciones cinemáticas del movimiento parabólico: $x(t) = x_0 + v_x \cdot t$ y $y(t) = y_0 + v_y \cdot t + 0.5 \cdot g \cdot t^2$, con un valor de gravedad simulada de 50 cm/s^2 . La función genera 15 puntos futuros a intervalos de 0.15 s que se dibujan en el canvas con color naranja para diferenciarlos de los objetos detectados.

Bibliotecas Utilizadas

Python (GUI y lógica de procesamiento)

- `tkinter`: biblioteca estándar de Python para construir interfaces gráficas. Se utilizó para la ventana del menú principal (`MenuPrincipal`), la pantalla del radar (`RadarGUI`) y todos los elementos visuales del HUD.
- `tkinter.font`: módulo de Tkinter para definir y cargar fuentes tipográficas personalizadas (Courier, distintos tamaños y pesos) usadas en el HUD del radar.
- `PIL / Pillow`: biblioteca de procesamiento de imágenes. Empleada para cargar la imagen de fondo del menú principal (`radar.png`) y aplicar ajustes de brillo mediante `ImageEnhance`.
- `math`: biblioteca estándar de Python. Usada para las conversiones trigonométricas (`math.cos`, `math.sin`, `math.radians`) necesarias en la conversión de coordenadas polares a cartesianas y en el cálculo de la predicción de trayectoria.
- `time`: biblioteca estándar. Utilizada para registrar timestamps de cada lectura del sensor, necesarios para el cálculo de velocidad ($v = d/t$).
- `threading`: biblioteca estándar de Python. Permite ejecutar la lectura serial en un hilo daemon independiente del hilo principal de la GUI, evitando que `readline()` bloquee el mainloop de Tkinter.
- `queue`: biblioteca estándar. Provee la clase `Queue`, que actúa como canal thread-safe entre el hilo de lectura serial (productor) y el loop de actualización de la GUI (consumidor).
- `serial (PySerial)`: biblioteca de terceros. Maneja la conexión y comunicación con el Arduino a través del puerto serial USB, incluyendo el escaneo de puertos disponibles mediante `serial.tools.list_ports`.
- `os`: biblioteca estándar. Utilizada para construir rutas de archivos de forma compatible entre sistemas operativos (`os.path.join`, `os.path.dirname`).

Arduino (firmware del hardware)

- `Servo.h`: biblioteca oficial del ecosistema Arduino. Permite controlar el ángulo del servomotor mediante la función `write()`, abstrayendo la generación de la señal PWM.
- Funciones nativas de Arduino: `pulseIn()` para medir el eco del HC-SR04, `digitalWrite()/delayMicroseconds()` para generar el pulso de trigger, `Serial.begin()/Serial.println()` para la comunicación con el computador.

Diagramas

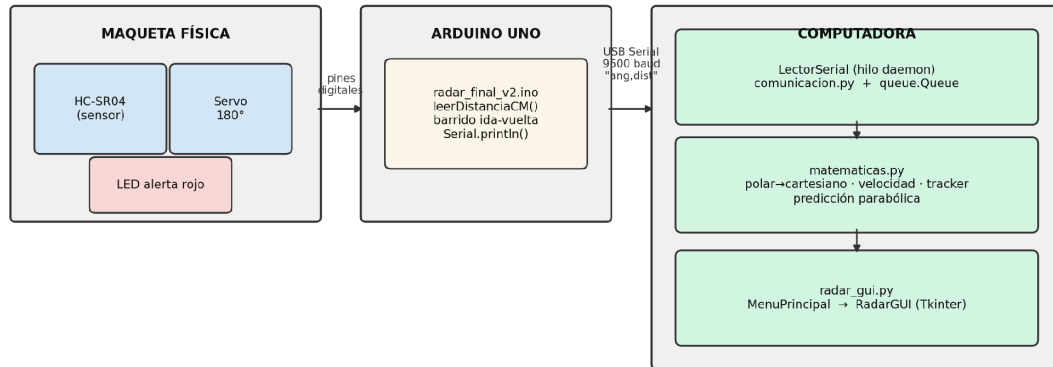


Figura 1. Diagrama de arquitectura básico — flujo de datos del Radar 2D

Figura 1. Diagrama de arquitectura básico

La arquitectura del sistema sigue una organización cliente-servidor donde el Arduino actúa como servidor de datos de sensado y el computador como cliente de procesamiento y visualización. El flujo es unidireccional: el hardware captura, el software procesa y muestra.

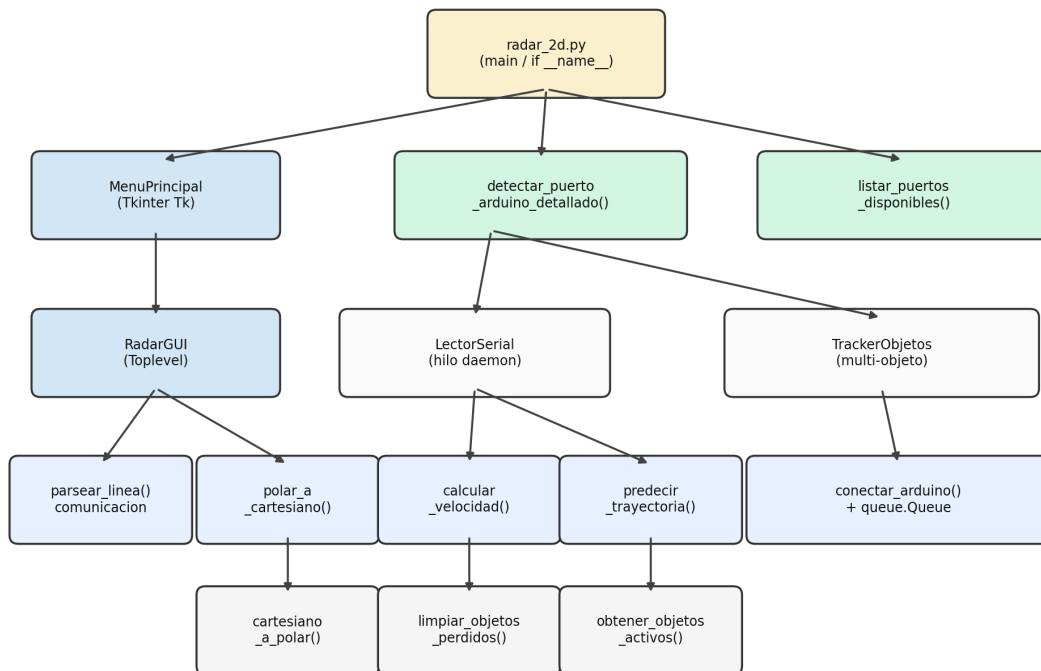


Figura 2. Diagrama de módulos del sistema

Figura 2. Diagrama de módulos del sistema

El diagrama de módulos refleja la jerarquía de dependencias del código Python. El punto de entrada (`if __name__ == '__main__'`) instancia `MenuPrincipal`, que al detectar el Arduino lanza `RadarGUI`. Esta a su vez utiliza `LectorSerial` para la comunicación y `TrackerObjetos` junto con las funciones matemáticas para el procesamiento de datos.

Fotos del sistema funcionando

[INSERTAR FOTO: Figura 3. Vista general de la maqueta física terminada (soldada, sin protoboard)]

Figura 3. Vista general de la maqueta física terminada (soldada, sin protoboard)

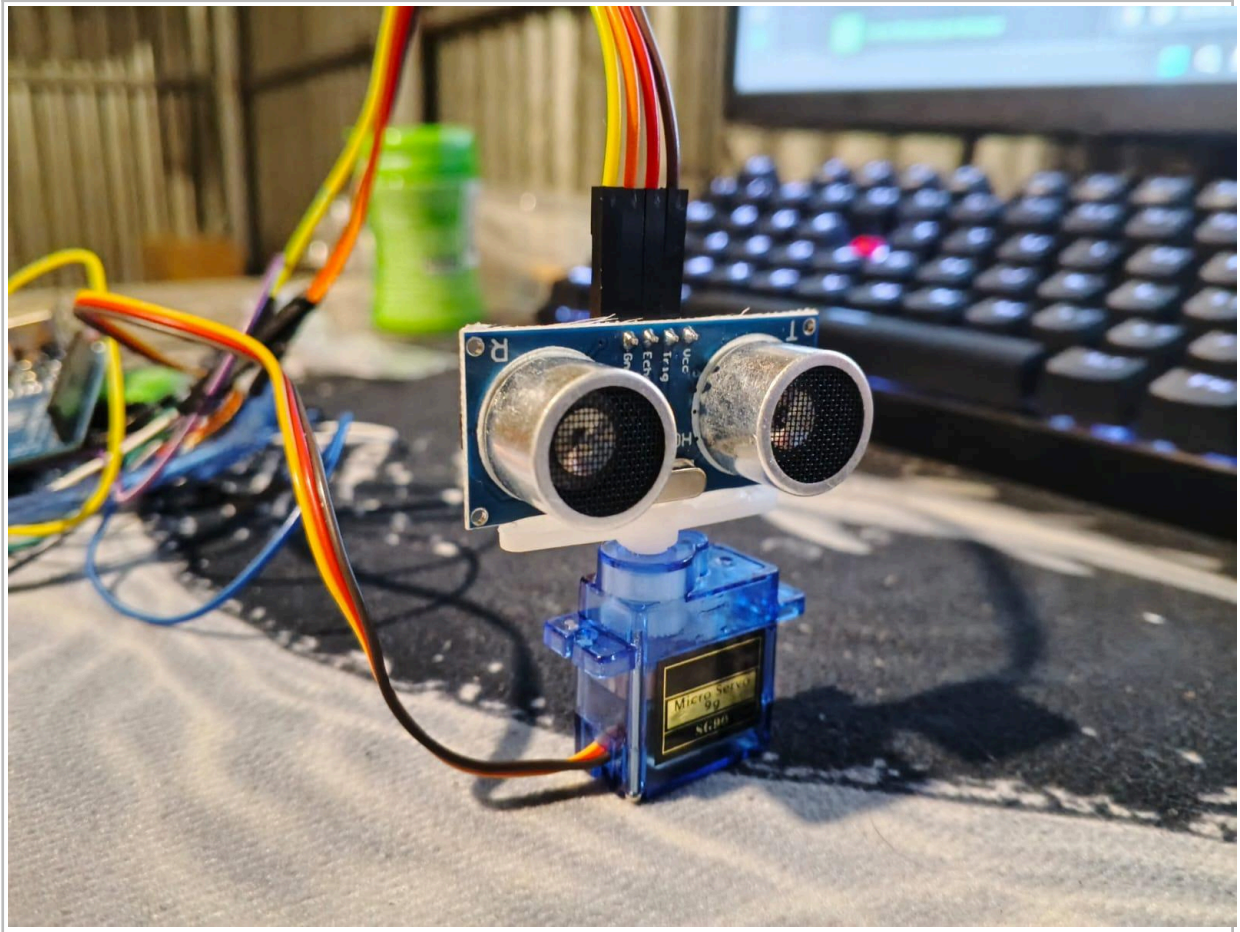


Figura 4. Detalle del sensor HC-SR04 montado sobre el servomotor

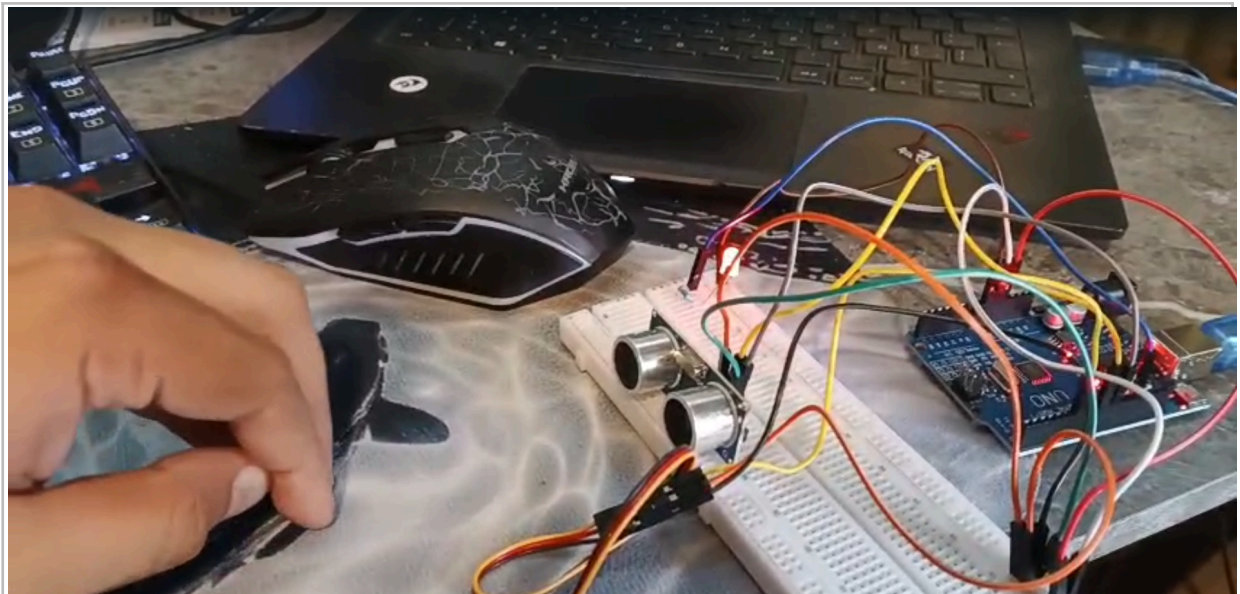


Figura 5. LED de alerta encendido al detectar objeto a menos de 15 cm

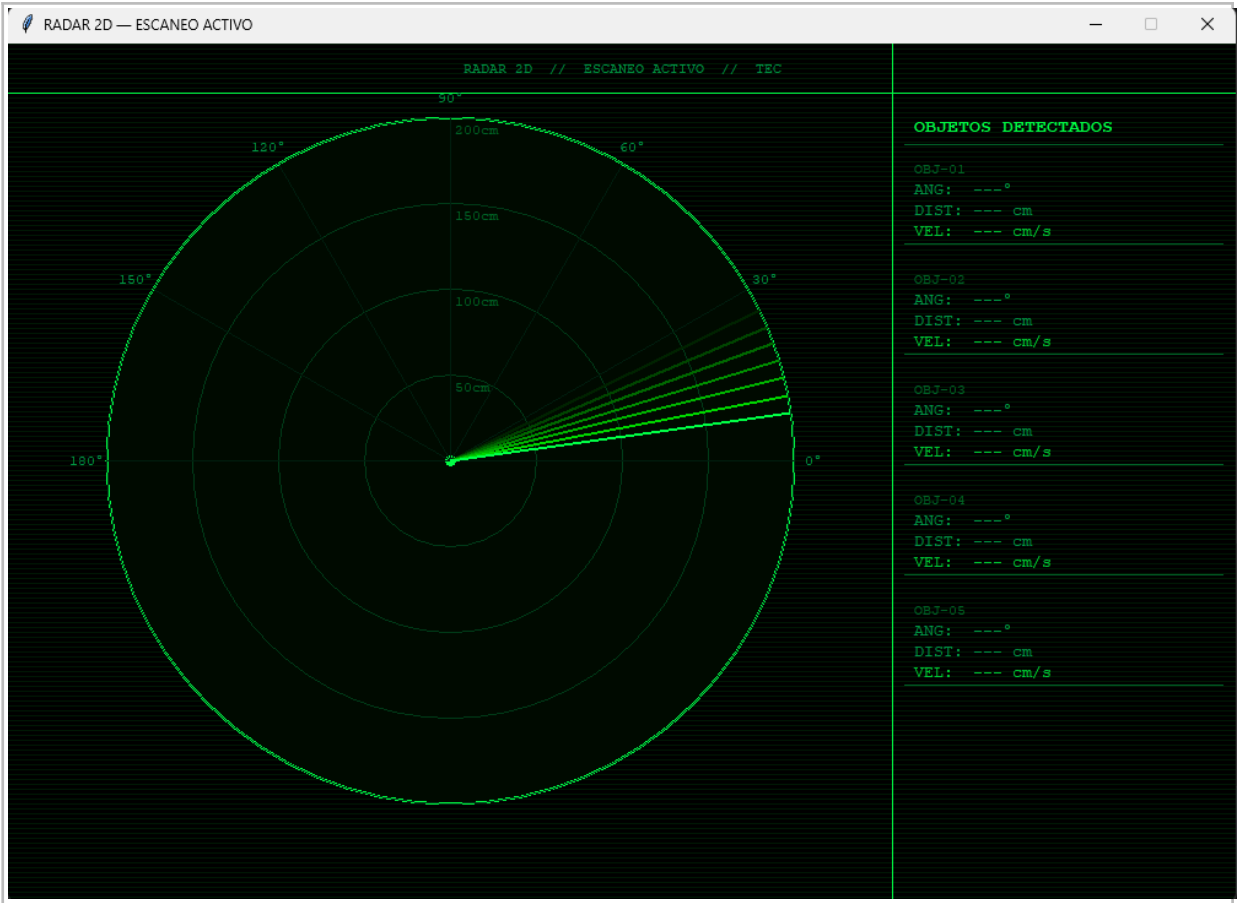


Figura 6. Interfaz gráfica del radar

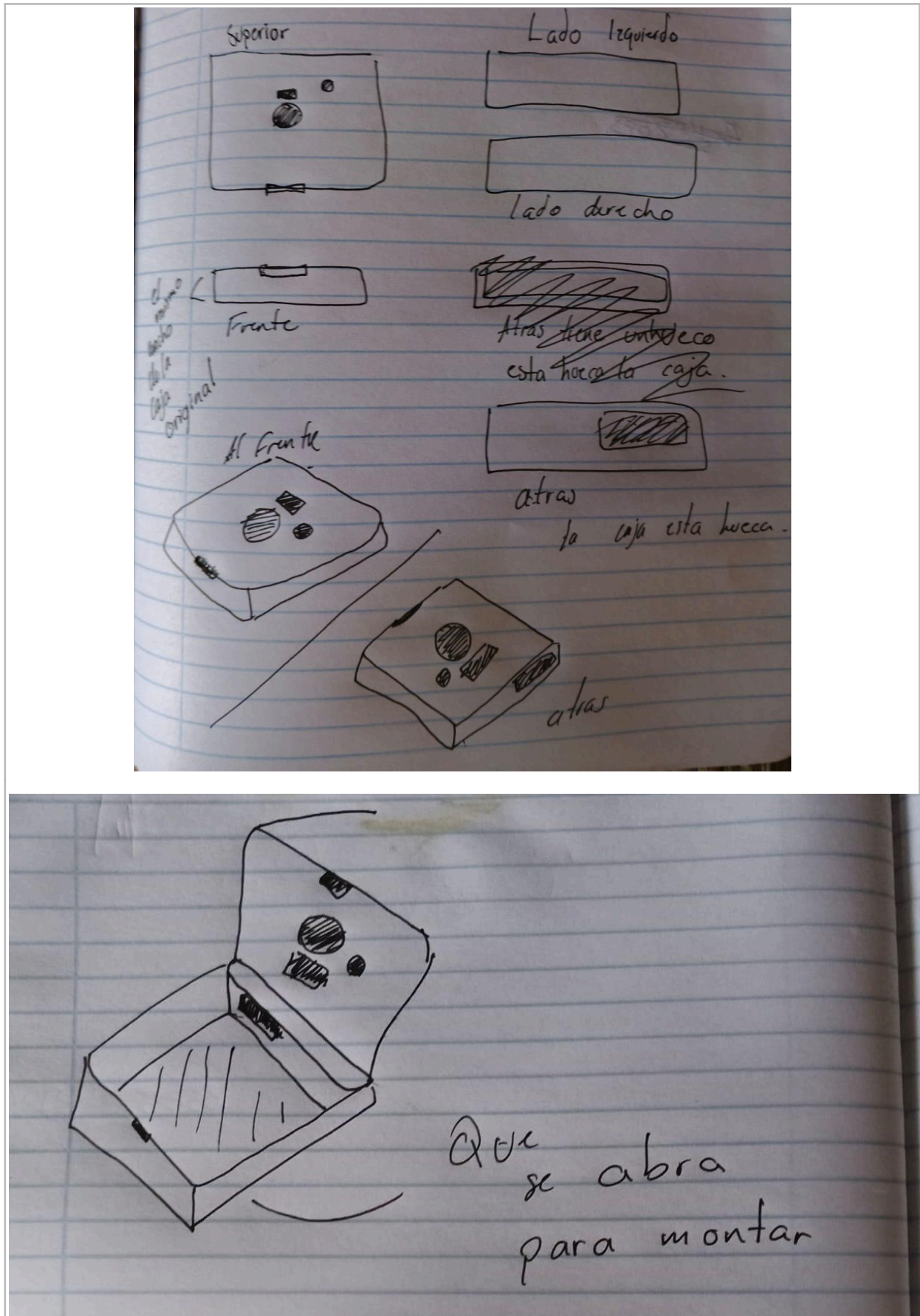


Figura 7 y 8. Bocetos de la maqueta

Referencias y Fuentes Consultadas

Desarrollo de Software e Interfaz Gráfica (Python / Tkinter)

[Documentación oficial de Python 3](#)

[Documentación oficial de Tkinter](#)

[PySerial — Comunicación serial en Python](#)

[PySerial Documentation \(readthedocs\)](#)

[Pillow — Python Imaging Library \(readthedocs\)](#)

Hardware y Arduino

[Referencia oficial de Arduino \(lenguaje y funciones\)](#)

[Biblioteca Servo.h — Documentación oficial Arduino](#)

[Datasheet HC-SR04 — Sensor ultrasónico de proximidad](#)

Diagramas y Diseño Lógico

[Draw.io / Diagrams.net — herramienta de diagramas gratuita](#)

Gestión del Proyecto y Control de Versiones

[GitHub — Repositorio del proyecto](#)

Otras referencias

<https://docs.arduino.cc/built-in-examples/arduino-isp/ArduinoISP/>

<https://aprendiendoarduino.wordpress.com/2017/01/23/programacion-arduino-5/>

https://www.youtube.com/watch?v=EEKMPT_YcTI

<https://www.contec.com/support/basic-knowledge/daq-control/serial-communicatin/>

<https://ccnadesdecero.es/conexion-serial-punto-a-punto/>

<https://prezi.com/p/yzu9303hki2d/calculo-de-posicion-y-velocidad-en-python/>

Referencias y Consultas con la IA

Durante el desarrollo del proyecto se consultó a herramientas de inteligencia artificial para resolver dudas puntuales. En todos los casos se solicitó orientación o pistas en lugar de respuestas directas, con el fin de preservar el proceso de aprendizaje. A continuación se describen las consultas más relevantes.

Chat 1 — Hilo serial y bloqueo de Tkinter

Yo: Tengo la interfaz en Tkinter y cuando intento leer el puerto serial la ventana se congela, ¿cómo lo evito?

IA: El problema es que `readline()` de `pyserial` es bloqueante y corre en el mismo hilo que el `mainloop` de Tkinter. La solución es mover la lectura serial a un `Thread` separado y comunicar los resultados al hilo principal mediante una `Queue thread-safe`. Investiga `threading.Thread` y `queue.Queue`, y cómo usar `after()` en Tkinter para consumir los datos periódicamente sin bloquear.

Chat 2 — Puerto serial ocupado por el IDE de Arduino

Yo: Conecto el Arduino, ejecuto mi programa en Python y me dice que el puerto no existe, pero en el IDE de Arduino sí lo veo.

IA: Dos programas no pueden usar el mismo puerto serial al mismo tiempo. Thonny o el IDE de Arduino mantienen la conexión abierta mientras están en uso. Cerrá el IDE antes de ejecutar tu script de Python. El puerto aparece en la esquina inferior del IDE, anotalo y usalo directamente en tu código.

Chat 3 — Detección automática del puerto Arduino

Yo: No quiero que el usuario tenga que escribir manualmente el puerto COM, ¿cómo puedo detectarlo automáticamente?

IA: `PySerial` incluye `serial.tools.list_ports.comports()` que lista todos los puertos activos junto con su descripción y fabricante. Podés filtrar esa lista buscando palabras clave como 'Arduino', 'CH340' o 'USB Serial' en los campos `description` y `manufacturer`. Los Arduinos clon chinos usan típicamente el chip CH340, así que incluí esa palabra clave en el filtro.

Chat 4 — Conversión de coordenadas polares a cartesianas

Yo: El Arduino me da el ángulo y la distancia, pero para dibujar en el canvas necesito (x, y). ¿Cómo los calculo?

IA: Usás las fórmulas de conversión polar a cartesiana: $x = r \cdot \cos(\theta)$ y $y = r \cdot \sin(\theta)$. Ojo que las funciones trigonométricas de Python trabajan en radianes, así que primero convertís el ángulo con `math.radians()`. Para el canvas de Tkinter recordá que el eje Y crece hacia abajo, por lo que quizás necesitéis invertir el signo de y para que la visualización sea correcta.